Lab A: Implementing Control Flow in an SSIS Package

Scenario

You are implementing an ETL solution for Adventure Works Cycles and must ensure that the data flows you have already defined are executed as a workflow that notifies operators of success or failure by sending an email message. You must also implement an ETL solution that transfers data from text files generated by the company's financial accounting package to the data warehouse.

Objectives

After completing this lab, you will be able to:

- Use tasks and precedence constraints.
- Use variables and parameters.
- Use containers.

Estimated Time: 60 minutes

Virtual machine: **20767C-MIA-SQL**

User name: **ADVENTUREWORKS\Student**

Password: **Pa55w.rd**

Exercise 1: Using Tasks and Precedence in a Control Flow

Scenario

You have implemented data flows to extract data and load it into a staging database as part of the ETL process for your data warehousing solution. Now you want to coordinate these data flows by implementing a control flow that notifies an operator of the outcome of the process.

The main tasks for this exercise are as follows:

1. Prepare the Lab Environment
2. View the Control Flow
3. Add Tasks to a Control Flow
4. Test the Control Flow

▶ Task 1: Prepare the Lab Environment

1. Ensure the **20767C-MIA-DC** and **20767C-MIA-SQL** virtual machines are both running, and then log on to **20767C-MIA-SQL** as **ADVENTUREWORKS\Student** with the password **Pa55w.rd**
2. Run **Setup.cmd** in the **D:\Labfiles\Lab07A\Starter** folder as Administrator.

▶ Task 2: View the Control Flow

1. Use **SQL Server Data Tools** to open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab07A\Starter\Ex1** folder.
2. Open the **Extract Reseller Data.dtsx** package and examine its control flow. Note that it contains two **Send Mail** tasks—one that runs when either the **Extract Resellers** or **Extract Reseller Sales** tasks fail, and one that runs when the **Extract Reseller Sales** task succeeds.
3. Examine the settings for the precedence constraint connecting the **Extract Resellers** task to the **Send Failure Notification** task to determine the conditions under which this task will be executed.

4. Examine the settings for the **Send Mail** tasks, noting that they both use the **Local SMTP Server** connection manager.
5. Examine the settings of the **Local SMTP Server** connection manager.
6. Run the package, and observe the control flow as the task executes.
7. When package execution is complete, stop debugging, and verify that the email message has been delivered to the **C:\inetpub\mailroot\Drop** folder. You can examine the email file by using Notepad, although the text of the message will be encoded.

▶ Task 3: Add Tasks to a Control Flow

1. Open the **Extract Internet Sales Data.dtsx** package and examine its control flow.
2. Add a **Send Mail** task to the control flow, configure it with the following settings, and create a precedence constraint that runs this task if the **Extract Internet Sales** task succeeds:
   - **Name**: Send Success Notification
   - **SmtpConnection**: A new SMTP Connection Manager named **Local SMTP Server** that connects to the **localhost** SMTP server
   - **From**: ETL@adventureworks.msft
   - **To**: Student@adventureworks.msft
   - **Subject**: Data Extraction Notification - Success
   - **MessageSourceType**: Direct Input
   - **MessageSource**: The Internet Sales data was successfully extracted
   - **Priority**: Normal
3. Add a second **Send Mail** task to the control flow, configure it with the following settings, and create a precedence constraint that runs this task if either the **Extract Customers** or **Extract Internet Sales** task fails:
   - **Name**: Send Failure Notification
   - **SmtpConnection**: The Local SMTP Server connection manager you created previously
   - **From**: ETL@adventureworks.msft
   - **To**: Student@adventureworks.msft
   - **Subject**: Data Extraction Notification - Failure
   - **MessageSourceType**: Direct Input
   - **MessageSource**: The Internet Sales data extraction process failed
   - **Priority**: High

▶ Task 4: Test the Control Flow

1. Set the **ForceExecutionResult** property of the **Extract Customers** task to **Failure**. Then run the package and observe the control flow.
2. When package execution is complete, stop debugging, and verify that the failure notification email message has been delivered to the **C:\inetpub\mailroot\Drop** folder. You can use Notepad to examine the email message and verify the subject line.

3. Set the **ForceExecutionResult** property of the **Extract Customers** task to **None**. Then run the package and observe the control flow.
4. When package execution is complete, stop debugging, and verify that the success notification email message has been delivered to the **C:\inetpub\mailroot\Drop** folder.
5. Close **SQL Server Data Tools** when you have completed the exercise.

Results: After this exercise, you should have a control flow that sends an email message if the **Extract Internet Sales** task succeeds, or sends an email message if either the **Extract Customers** or **Extract Internet Sales** tasks fail.

Exercise 2: Using Variables and Parameters

Scenario

You need to enhance your ETL solution to include the staging of payments data that is generated in comma-separated value (CSV) format from a financial accounts system. You have implemented a simple data flow that reads data from a CSV file and loads it into the staging database. You must now modify the package to construct the folder path and file name for the CSV file dynamically at run time instead of relying on a hard-coded name in the Data Flow task settings.

The main tasks for this exercise are as follows:

1. View the Control Flow
2. Create a Variable
3. Create a Parameter
4. Use a Variable and a Parameter in an Expression

▶ Task 1: View the Control Flow

1. View the contents of the **D:\Accounts** folder and note the files it contains. In this exercise, you will modify an existing package to create a dynamic reference to one of these files.
2. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab07A\Starter\Ex2** folder.
3. Open the **Extract Payment Data.dtsx** package and examine its control flow. Note that it contains a single Data Flow task named **Extract Payments**.
4. View the **Extract Payments** data flow and note that it contains a flat file source named **Payments File**, and an OLE DB destination named **localhost.Staging** This destination connects to the Staging database.
5. View the settings of the **Payments File** source and note that it uses a connection manager named **Payments File**.
6. In the Connection Managers pane, double-click **Payments File**, and note that it references the Payments.csv file in the **D:\Labfiles\Lab07A\Starter\Ex2** folder. This file has the same data structure as the payments file in the **D:\Accounts** folder.
7. Run the package, and stop debugging when it has completed.
8. On the **Execution Results** tab, find the following line in the package execution log:

   [Payments File [2]: The processing of the file "D:\Labfiles\Lab07A\Starter\Ex2\Payments.csv" has started

▶ Task 2: Create a Variable

- Add a variable with the following properties to the package:
  - **Name**: fName
  - **Scope**: Extract Payment Data
  - **Data type**: String
  - **Value**: Payments - US.csv

  Note that the Value property includes a space on either side of the "-" character.

▶ Task 3: Create a Parameter

- Add a project parameter with the following settings:
  - **Name**: AccountsFolderPath
  - **Data type**: String
  - **Value**: D:\Accounts\
  - **Sensitive**: False
  - **Required**: True
  - **Description**: Path to accounts files

Note: Be sure to include the trailing "\" in the Value property.

▶ Task 4: Use a Variable and a Parameter in an Expression

1. Set the **Expressions** property of the **Payments File** connection manager in the **Extract Payment Data** package so that the **ConnectionString** property uses the following expression:

   @[$Project::AccountsFolderPath]+ @[User::fName]

2. Run the package and view the execution results to verify that the data in the **D:\Accounts\Payments - US.csv** file was loaded.
3. Close **SQL Server Data Tools** when you have completed the exercise.

Results: After this exercise, you should have a package that loads data from a text file based on a parameter that specifies the folder path where the file is stored, and a variable that specifies the file name.

Exercise 3: Using Containers

Scenario

You have created a control flow that loads Internet sales data and sends a notification email message to indicate whether the process succeeded or failed. You now want to encapsulate the Data Flow tasks for this control flow in a Sequence container so you can manage them as a single unit.

You have also successfully created a package that loads payments data from a single CSV file, based on a dynamically derived folder path and file name. Now you must extend this solution to iterate through all the files in the folder and import data from each one.

The main tasks for this exercise are as follows:

1. Add a Sequence Container to a Control Flow
2. Add a Foreach Loop Container to a Control Flow

▶ Task 1: Add a Sequence Container to a Control Flow

1. Open the **AdventureWorksETL** solution in the **D:\Labfiles\Lab07A\Starter\Ex3** folder.
2. Open the **Extract Internet Sales Data.dtsx** package and modify its control flow so that:
   - The **Extract Customers** and **Extract Internet Sales** tasks are contained in a Sequence container named **Extract Customer Sales** Data.
   - The **Send Failure Notification** task is executed if the **Extract Customer Sales Data** container fails.
   - The **Send Success Notification** task is executed if the **Extract Customer Sales Data** container succeeds.
3. Run the package to verify that it successfully completes both Data Flow tasks in the sequence, and then executes the **Send Success Notification** task.

▶ Task 2: Add a Foreach Loop Container to a Control Flow

1. In the **AdventureWorksETL** solution, open the **Extract Payment Data.dtsx** package.
2. Move the existing **Extract Payment** Data Flow task into a new **Foreach Loop Container**.
3. Configure the **Foreach Loop Container** so that it loops through the files in the folder referenced by the **AccountsFolderPath** parameter, adding each file to the **fName** variable.
4. Run the package and count the number of times the **Foreach Loop** is executed.
5. When execution has completed, stop debugging and view the results to verify that all files in the **D:\Accounts** folder were processed.
6. Close **SQL Server Data Tools** when you have completed the exercise.

Results: After this exercise, you should have one package that encapsulates two Data Flow tasks in a Sequence container, and another that uses a Foreach Loop to iterate through the files in a folder specified in a parameter—and uses a Data Flow task to load their contents into a database.